

CURSO

TEST AUTOMATION ENGINEER

FORMACIÓN INTEGRAL



Objetivo General del Curso

ANALIZAR LAS HERRAMIENTAS DE DISEÑO DE TEST AUTOMATION ENGINEER, DE ACUERDO A LAS APLICACIONES WEB, MÓVILES Y APIS.

Objetivo específico del Módulo

ANALIZAR LAS HERRAMIENTAS DE AUTOMATIZACIÓN DE UN FLUJO COMPLETO WEB API CI/CD, DE ACUERDO A LAS APLICACIONES WEB, MÓVILES Y APIS.

Contenidos

Objetivo General del Curso	2
Objetivo específico del Módulo.....	2
Módulo 12: HERRAMIENTAS DE AUTOMATIZACIÓN DE UN FLUJO COMPLETO WEB + API + CICD	4
Capítulo 1: AUTOMATIZACIÓN DE UN FLUJO COMPLETO WEB + API + CI/CD	5
Componentes del Flujo Automatizado	5
Pruebas Automatizadas de API.....	6
Automatización en CI/CD	7
Buenas Prácticas	10
Capítulo 2: ORGANIZACIÓN Y DOCUMENTACIÓN DEL REPOSITORIO	11
Documentación Técnica del Repositorio	13
Desglose y explicación de un ejemplo práctico	15
Buenas prácticas para workflows CI/CD	18
Capítulo 3: VIDEO DE PRESENTACIÓN DEL PROYECTO.....	19
Requisitos técnicos mínimos	19
Contenidos obligatorios:	19
Criterios de evaluación:	19
Capítulo 4: RÚBRICA DE EVALUACIÓN TÉCNICA Y FEEDBACK CUALITATIVO.....	20
Feedback cualitativo.....	20
Capítulo 5: EMISIÓN DE CERTIFICADO OFICIAL DE FINALIZACIÓN	21
Requisitos para obtener el certificado:	21
Contenido del certificado:	21
Formato de entrega:	21

Módulo 12: HERRAMIENTAS DE AUTOMATIZACIÓN DE UN FLUJO COMPLETO WEB + API + CICD



Capítulo 1: AUTOMATIZACIÓN DE UN FLUJO COMPLETO WEB + API + CI/CD

La automatización de un flujo completo en un entorno de desarrollo moderno comprende la validación de distintos niveles del sistema: interfaz de usuario (web), lógica de negocio (API), y procesos de integración y despliegue continuo (CI/CD). Este enfoque permite mantener la calidad del software, disminuir errores en producción y acelerar los ciclos de entrega.

El presente capítulo aborda el proceso de integración de pruebas automatizadas en un flujo continuo de desarrollo y despliegue, utilizando herramientas especializadas que permiten orquestar todo el proceso desde el código fuente hasta el ambiente de producción o staging.

Componentes del Flujo Automatizado

Pruebas Automatizadas de la Interfaz Web

Las pruebas de interfaz gráfica se enfocan en simular la interacción del usuario con el sistema. Se validan aspectos como la navegación, formularios, estados visuales y mensajes.

Herramientas comunes: Cypress, Playwright, Selenium.

Ejemplo de flujo de prueba:

- Acceder a la página de login.
- Ingresar credenciales.
- Verificar redirección a panel de usuario.
- Validar la visibilidad de elementos clave.

Fragmento de código en Cypress:

```
javascript

describe('Prueba de acceso al panel', () => {
  it('Debería iniciar sesión correctamente', () => {
    cy.visit('/login');
    cy.get('#usuario').type('usuario_testeo');
    cy.get('#clave').type('123456');
    cy.get('form').submit();
    cy.url().should('include', '/panel');
    cy.contains('Bienvenido').should('be.visible');
  });
});
```

Pruebas Automatizadas de API

Las pruebas de servicios API permiten validar la lógica de negocio, operaciones de base de datos, respuestas de backend y cumplimiento de especificaciones REST o GraphQL.

Herramientas comunes: Postman, Newman, Rest Assured, Supertest.

Pruebas típicas:

- Verificación de códigos de estado HTTP.
- Validación de estructuras JSON.
- Pruebas de carga mínima y tiempos de respuesta.

Ejemplo de ejecución con Newman desde CLI:

```
bash
```

```
newman run tests/postman_collection.json -e entorno.json --reporters cli,html
```

Automatización en CI/CD

La automatización continua se integra en pipelines que ejecutan todas las etapas del ciclo de vida: compilación, pruebas, revisión de calidad y despliegue.

Herramientas comunes: GitHub Actions, GitLab CI, Jenkins, CircleCI.

Etapas comunes de un pipeline:

- Clonación del repositorio.
- Instalación de dependencias.
- Ejecución de pruebas unitarias.
- Ejecución de pruebas UI y API.
- Análisis de calidad de código.
- Despliegue automático (si todas las pruebas pasan).

Ejemplo de archivo de configuración para GitHub Actions:

Archivo: .github/workflows/flujo-completo.yml

```
yml
name: Flujo completo de pruebas

on: [push]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Clonar código
        uses: actions/checkout@v3

      - name: Configurar Node.js
        uses: actions/setup-node@v3
        with:
          node-version: '18'

      - name: Instalar dependencias
        run: npm install

      - name: Ejecutar pruebas Cypress
        run: npm run cy:run

      - name: Ejecutar pruebas API con Newman
        run: |
          npm install -g newman
          newman run ./tests/api/postman_collection.json
```


name: Flujo completo de pruebas

on: [push]

jobs:

test:

runs-on: ubuntu-latest

steps:

- name: Clonar código

uses: actions/checkout@v3

- name: Configurar Node.js

uses: actions/setup-node@v3

with:

node-version: '18'

- name: Instalar dependencias

run: npm install

- name: Ejecutar pruebas Cypress

run: npm run cy:run

- name: Ejecutar pruebas API con Newman

run: |

npm install -g newman

newman run ./tests/api/postman_collection.json

Buenas Prácticas

- Separar pruebas por tipo: Evita mezclar pruebas unitarias, de integración y de interfaz.
- Usar datos de prueba aislados: Para prevenir interferencias entre ejecuciones.
- Habilitar reportes automáticos: Preferir reportes en HTML o integraciones con dashboards.
- Fail fast: Detener el pipeline si alguna prueba crítica falla.
- Reintentar pruebas inestables: Configurar retries en pruebas UI si son frágiles.
- Utilizar entornos controlados o mockeados para pruebas API.

La automatización de un flujo completo que abarca interfaz web, servicios API y procesos CI/CD representa un componente fundamental en cualquier estrategia de aseguramiento de calidad moderna. Su implementación requiere una combinación de habilidades técnicas, uso adecuado de herramientas y una visión clara del proceso de desarrollo. Este enfoque permite no solo detectar errores de forma temprana, sino también mantener la integridad del software en ciclos de entrega cada vez más cortos y exigentes.

Capítulo 2: ORGANIZACIÓN Y DOCUMENTACIÓN DEL REPOSITORIO

Una correcta organización y documentación del repositorio de pruebas automatizadas es fundamental para garantizar la mantenibilidad, escalabilidad, trazabilidad y colaboración efectiva entre los miembros del equipo de desarrollo, QA y DevOps. Un repositorio bien estructurado permite a cualquier persona entender la lógica del proyecto, ejecutar las pruebas sin dificultades y aportar mejoras sin introducir errores.

Este apartado entrega las recomendaciones técnicas y estructurales para organizar un repositorio orientado a pruebas automatizadas en flujos Web, API y CI/CD.

2. Estructura Recomendada del Repositorio

A continuación, se presenta una estructura típica para un proyecto de automatización que integra pruebas de frontend (UI), backend (API), y configuración de CI/CD.

```
bash

/proyecto-automatizacion/
├── tests/
│   ├── unit/           # Pruebas unitarias
│   ├── integration/    # Pruebas de integración
│   ├── e2e/           # Pruebas end-to-end (web UI)
│   └── api/            # Pruebas de API (Postman, Supertest, etc.)
├── data/
│   ├── fixtures/       # Datos de prueba (JSON, CSV, etc.)
│   └── environment/    # Configuraciones por entorno (dev, qa, prod)
├── reports/            # Reportes generados (HTML, XML, JSON)
├── scripts/            # Scripts utilitarios (instalación, ejecución)
├── .github/            # Configuración de Github Actions (o .gitlab-ci.yml)
│   └── workflows/
├── docs/               # Documentación técnica (Markdown, diagramas)
├── .env                # Variables de entorno locales
├── README.md           # Descripción del proyecto
├── package.json        # Dependencias y comandos NPM (si usa Node.js)
└── config.js           # Configuración global del framework de pruebas
```

/proyecto-automatizacion/

- |
 - |— tests/
 - | |— unit/ # Pruebas unitarias
 - | |— integration/ # Pruebas de integración
 - | |— e2e/ # Pruebas end-to-end (web UI)
 - | |— api/ # Pruebas de API (Postman, Supertest, etc.)
 - |— data/
 - | |— fixtures/ # Datos de prueba (JSON, CSV, etc.)
 - | |— environment/ # Configuraciones por entorno (dev, qa, prod)
 - |— reports/ # Reportes generados (HTML, XML, JSON)
 - |— scripts/ # Scripts utilitarios (instalación, ejecución)
 - |— .github/ # Configuración de GitHub Actions (o .gitlab-ci.yml)
 - | |— workflows/
 - |— docs/ # Documentación técnica (Markdown, diagramas)
 - |— .env # Variables de entorno locales
 - |— README.md # Descripción del proyecto
 - |— package.json # Dependencias y comandos NPM (si usa Node.js)
 - |— config.js # Configuración global del framework de pruebas

Documentación Técnica del Repositorio

Archivo README.md

El archivo README.md debe ser el primer punto de contacto para quien acceda al repositorio. Debe contener:

- Nombre del proyecto y propósito general.
- Requisitos del entorno (Node.js, Java, dependencias, etc.).
- Instrucciones de instalación.
- Comandos de ejecución de pruebas.
- Estructura del proyecto (descripción de carpetas principales).
- Instrucciones para integración CI/CD.
- Créditos y enlaces útiles.

Ejemplo de README.md:

```
## Automatización QA Proyecto XYZ

Este repositorio contiene la suite de pruebas automatizadas para el sistema XYZ, incluyendo pruebas de interfaz web, servicios API, y flujos CI/CD.

## Requisitos

- Node.js v18
- npm
- Git
- Docker (optional)

## Instalación

```bash
git clone https://github.com/empresa/proyecto-automatizacion.git
cd proyecto-automatizacion
npm install
```

# Automatización QA Proyecto XYZ

Este repositorio contiene la suite de pruebas automatizadas para el sistema XYZ, incluyendo pruebas de interfaz web, servicios API, y flujos CI/CD.

## ## Requisitos

- Node.js v18
- npm
- Git
- Docker (opcional)

## ## Instalación

```
```bash
git clone https://github.com/empresa/proyecto-automatizacion.git
cd proyecto-automatizacion
npm install
```

Ejecución

- Pruebas UI: `npm run test:e2e`
- Pruebas API: `newman run ./tests/api/coleccion.postman.json`

Integración CI/CD

Ver configuración en `.github/workflows/flujo-completo.yml`

El archivo `.github/workflows/flujo-completo.yml` forma parte de la integración continua (CI) y entrega continua (CD) del proyecto. Este archivo define el flujo automatizado que se ejecuta cada vez que ocurre un evento en el repositorio (por ejemplo, un *push*, *pull request* o *release*). Su objetivo es garantizar que todo el ciclo de construcción, prueba y despliegue de la aplicación esté controlado y automatizado.

GitHub Actions permite definir estos flujos mediante archivos YAML, que especifican los pasos secuenciales a ejecutar en máquinas virtuales preconfiguradas. En este caso, el flujo cubre pruebas automatizadas para interfaz web y API, dentro de un pipeline reproducible y versionado.

2. Ubicación y estructura del archivo

El archivo se debe ubicar en la siguiente ruta del repositorio:

```
bash

/.github/
├─ workflows/
│   └─ flujo-completo.yml
```

Esta es la ubicación estándar utilizada por **GitHub Actions** para detectar y ejecutar workflows definidos por el usuario.

Desglose y explicación de un ejemplo práctico

A continuación, se muestra un archivo de configuración de ejemplo, con cada sección explicada detalladamente.

name: Flujo completo de pruebas automatizadas

on:

push:

branches:

- main

pull_request:

branches:

- main

jobs:

test:

runs-on: ubuntu-latest

steps:

- name: Clonar repositorio

uses: actions/checkout@v3

- name: Configurar Node.js

uses: actions/setup-node@v3

with:

node-version: '18'

- name: Instalar dependencias
run: npm install
- name: Ejecutar pruebas UI (Cypress)
run: npm run test:e2e
- name: Ejecutar pruebas API (Newman)
run: |
npm install -g newman
newman run ./tests/api/postman_collection.json --reporters cli,html
- name: Generar artefactos de reporte
uses: actions/upload-artifact@v3
with:
name: reporte-pruebas
path: reports/

Explicación de cada sección

name

Define el nombre del workflow, visible en la pestaña Actions de GitHub.

name: Flujo completo de pruebas automatizadas

on

Define los eventos que activan el workflow. En este caso, se ejecuta:

Al hacer push en la rama main.

Al crear un pull_request hacia la rama main.

on:

push:

branches:

- main

pull_request:

branches:

- main

jobs

Define los trabajos a ejecutar. En este ejemplo, hay un solo trabajo llamado test, ejecutado en una máquina virtual con Ubuntu.

jobs:

test:

runs-on: ubuntu-latest

steps

Lista las acciones dentro del trabajo test:

1.- Clonación del repositorio:

- name: Clonar repositorio

uses: actions/checkout@v3

2.- Configuración del entorno de ejecución (Node.js):

- name: Configurar Node.js

uses: actions/setup-node@v3

with:

node-version: '18'

3.- Instalación de dependencias:

- name: Instalar dependencias

run: npm install

4.- Ejecución de pruebas de UI (Cypress o similar):

- name: Ejecutar pruebas UI (Cypress)

run: npm run test:e2e

5.- Ejecución de pruebas de API con Postman (Newman):

- name: Ejecutar pruebas API (Newman)

run: |

npm install -g newman

newman run ./tests/api/postman_collection.json --reporters cli,html

6.- Carga de reportes como artefactos del workflow:

- name: Generar artefactos de reporte
uses: actions/upload-artifact@v3
with:
 name: reporte-pruebas
 path: reports/

Buenas prácticas para workflows CI/CD

- Fail fast: el workflow debe detenerse en el primer error crítico.
- Paralelización: separar pruebas por tipo (UI, API, integración) en jobs diferentes si se desea mayor velocidad.
- Persistencia de reportes: guardar reportes como artefactos o integrarlos con dashboards externos (Allure, ReportPortal).
- Variables de entorno seguras: usar secrets de GitHub para manejar tokens o claves de entorno.
- Etiquetado de versiones: automatizar releases si las pruebas son exitosas.

La configuración del archivo `.github/workflows/flujo-completo.yml` permite establecer una automatización robusta para validar cada cambio de código mediante pruebas automatizadas. Este enfoque mejora la eficiencia del equipo, reduce el error humano, y asegura un flujo de entrega continuo alineado con los principios de DevOps y calidad de software.

Capítulo 3: VIDEO DE PRESENTACIÓN DEL PROYECTO

El video de presentación busca que usted demuestre no solo el resultado técnico de su trabajo, sino también su capacidad para explicar, justificar y comunicar el proceso completo de automatización desarrollado. Es una instancia clave para evaluar habilidades de documentación audiovisual y síntesis profesional.

Requisitos técnicos mínimos

- Formato: MP4 o enlace privado (YouTube, Vimeo, Google Drive).
- Duración: entre 3 a 7 minutos.
- Herramientas sugeridas: OBS Studio, Loom, ScreenPal, Zoom grabado.

Contenidos obligatorios:

- Descripción general del sistema automatizado.
- Breve explicación de la estructura del repositorio.
- Demostración en tiempo real de ejecución de pruebas Web + API.
- Explicación de la configuración del pipeline CI/CD.
- Menció de herramientas utilizadas.
- Dificultades enfrentadas y cómo se resolvieron.

Criterios de evaluación:

- Claridad y estructura del discurso.
- Calidad visual y técnica de la grabación.
- Coherencia entre lo presentado y lo desarrollado en el repositorio.

Capítulo 4: RÚBRICA DE EVALUACIÓN TÉCNICA Y FEEDBACK CUALITATIVO

Objetivo

Establecer criterios objetivos para calificar el proyecto final, permitiendo retroalimentación constructiva basada en evidencias técnicas. La rúbrica debe estar alineada con los objetivos de aprendizaje del curso.

Ejemplo de Rúbrica Técnica

Criterio	Nivel 1 (0 pts)	Nivel 2 (1 pt)	Nivel 3 (2 pts)	Nivel 4 (3 pts)
Organización del repositorio	Desorganizado	Parcialmente ordenado	Bien estructurado	Estructura profesional
Pruebas Web automatizadas	No presentes	Básicas y limitadas	Completas	Complejas y optimizadas
Pruebas API automatizadas	No presentes	Básicas	Completas	Con manejo de datos y errores
Integración CI/CD funcional	No implementada	Parcial	Funcional básica	Flujo completo validado
Calidad del código y documentación	Caótica o sin documentar	Parcialmente documentado	Código limpio y comentado	Código reusable y documentado
Presentación del proyecto (video)	No entregado	Entregado sin estructura	Clara, estructurada	Profesional y convincente

Puntuación máxima:	18 puntos
Nivel de logro:	
0-5:	Insuficiente
6-11:	Aceptable
12-15:	Bueno
16-18:	Excelente

Feedback cualitativo

Se debe entregar un informe personalizado que contenga:

- Puntos fuertes del trabajo.
- Áreas de mejora.
- Recomendaciones para escenarios reales de automatización.
- Comentarios sobre la evolución del participante en el curso.

Capítulo 5: EMISIÓN DE CERTIFICADO OFICIAL DE FINALIZACIÓN

Requisitos para obtener el certificado:

- Haber completado al menos el 80% de los módulos teóricos del curso.
- Haber entregado el proyecto completo con:
- Repositorio organizado y documentado.
- Video de presentación.
- Ejecución de pruebas automatizadas funcionales.
- Haber obtenido una calificación mínima de 12 puntos sobre 18 en la rúbrica.

Contenido del certificado:

- Nombre completo del participante.
- Nombre del curso: Test Automation Engineer – Web + API + CI/CD.
- Duración: horas cronológicas totales del curso (ej. 60 horas).
- Firma del instructor y/o institución certificadora.
- Fecha de finalización.

Formato de entrega:

- PDF digital con código único de verificación.
- Opción de impresión a solicitud.